



— MY PERSONAL C NOTEBOOK • DIGITIZED —

C PROGRAMMING

My handwritten notes on C — from first principles to pointers, structures, files, and dynamic memory — kept exactly the way I learned them.

CHAPTER 0 – SETUP

VARIABLES

LOOPS

FUNCTIONS

POINTERS

ARRAYS

STRINGS

STRUCTURES

FILE I/O

DMA

MADE BY AVIJIT BISWAS

— WHITE DEV —

HANDWRITTEN NOTES, DIGITIZED FOR REVISION

Table of Contents

My own notebook, organized chapter by chapter — exactly how I studied it.

0	Setup & First Steps	Compiler, VS Code, MinGW, first program
1	Variables & Data Types	int/float/char, printf, scanf, comments
2	Instructions & Operators	Precedence, type conversion, typecasting
3	Conditions & Logic	Relational/logical operators, ternary, switch
4	Loops	while, do-while, for, break/continue
5	Functions & Recursion	Prototype, parameters, base case, cheat sheet
6	Pointers	The house & address analogy, call by reference
7	Arrays	Indexing, memory, pointer arithmetic, 2-D
8	Strings	Null character, the Fantastic Four functions
9	Structures	struct, dot/arrow operator, typedef
10	File I/O	fopen modes, fscanf/fprintf, fgetc/fputc
11	Dynamic Memory Allocation	malloc, calloc, realloc, free

Setup & First Steps



A programming language is a way for us to deliver instructions to a computer, the same way we use a language to communicate with each other. **C** is one of the oldest and finest programming languages — created by Dennis Ritchie at Bell Labs in 1972.

What is a Compiler?

A compiler is a tool that converts high-level code (like C) into low-level machine/binary code that a computer can actually read.

• Setting Up VS Code

- Install VS Code, then install the **C/C++** extension and the **Code Runner** extension from the Extensions panel.
- These extensions add intelligent code suggestions and let you run C code directly inside VS Code.

• Compiler Installation (MinGW)

1. Go to `MinGW.com` → click **Download** → keep clicking **Continue**.
2. Choose the `gcc-with-c++` option only → go to **Action** (top-left) → click **Apply Changes**. This takes a little time.
3. After installing, open your **C drive** → find the `MinGW` folder → open it → open `bin`. You'll see `gcc` there. Copy the folder path (URL) from the address bar.

SETUP STEPS

Search "**environment variables**" in Windows search → click **Edit environment variables** → click **Path** → click **New** → paste the `bin` path you copied → click **OK** → **OK** → **OK**.

Then close VS Code and open it again.

- ✓ **How to check it worked:** Open a terminal and type `gcc` then press Enter. If it shows a "**fatal error**", the installation is correct! If it shows something like "unrecognised command", redo the environment variable step above.

• Running Your First Program

```
#include <stdio.h>

int main() {
    printf("Hello World");
    return 0;
}
```

Save this file as `01_first.c`. Open the terminal inside VS Code (`ctrl + ``) and type:

```
gcc 01_first.c
```

This creates a new file called `a.exe` — this means gcc has successfully converted your code into computer code. To actually run it and see the output, type:

```
./a.exe
```

You should now see "**Hello World**" printed in the terminal.

- ★ To skip typing these two commands every time, install the **Code Runner** extension — it lets you run a file with a single click (the ▶ button top-right).

Variable (রাখার জায়গা)

A container that holds a value.

```
int a = 1;
// or
int a;
a = 1;
```

Value vs Integer

Value → রাখার ভেতর যা আছে (what's stored inside).

Integer → রাখার ভেতর কি ধরনের বস্তু আছে (what type of thing is stored).

In `int a = 1;` → **a** is the variable, **1** is the value.

The Three Basic Data Types

Type	Stores	Format Specifier	Size
int	Only whole numbers	%d	4 bytes
float	Only decimal numbers	%f	4 bytes
char	A single character	%c	1 byte

These sizes decide how much space (RAM) a variable takes up.

✔ **Single vs multiple characters:** `%c` is for a **single character**, but `%s` is for **multiple characters** (a string, covered in Chapter 8).

Naming Rules for Variables

- Only letters (a–z, A–Z), digits, and underscore `_` are allowed.
- No spaces allowed anywhere in a variable name.
- Names are case-sensitive.

Comments

Single-line

```
// comment
```

Multi-line

```
/* comment */
```

Taking Input — scanf

If we don't fix a value ourselves, we let the user type it in:

```
scanf("%d", &a);
```

যে ভ্যারিয়েবলে আমরা ইনপুট রাখতে চাই, তার আগে `&` বসাতে হয় | The format specifier must match the variable's type — `%d` for int, `%f` for float, `%c` for char.

Escape Sequences

Sequence	Meaning
<code>\n</code>	New line
<code>\t</code>	Tab space
<code>\\</code>	A single backslash <code>\</code>

• Three Types of Instructions

#	Instruction Type	What it means
1	Type declaration	Just the normal type declarations we do regularly (<code>int a;</code> etc.)
2	Arithmetic instruction	Mainly used for mathematical terms/operations
3	Control instruction	Controls the flow of the program (see below)

★ **4 types of control instructions:** Sequence, Decision, Loop, and Case control instructions.

• Operators, Operands & Results

`x op y = z` — here x and y are **operands**, op is the **operator**, and z is the **result**.

{ `+`, `-`, `*`, `/`, `%` } → these are the **arithmetic operators**.

! **Correction:** C's operator precedence doesn't literally follow school BODMAS (there's no bracket/order step built in), but its own precedence table gives the same practical result for arithmetic: `*`, `/`, `%` are evaluated first, then `+`, `-`, then `=` last.

• C's Order of Preference

Priority	Operators
1st choice	<code>*</code> <code>/</code> <code>%</code>
2nd choice	<code>+</code> <code>-</code>
3rd choice	<code>=</code>

So if an equation has `+`, `=`, `-`, `*`, `/`, C first takes `(*, /)`, then `(+, -)`, and lastly `(=)`.

✓ **When operators of the same priority appear together** (e.g. `*`, `/`, `%` all in one equation), C resolves them **left to right**. Example order: `*` → `/` → `%` as they appear left to right.

• Type Conversion

Operation	Result Type
<code>int + int</code>	<code>int</code>
<code>int + float</code>	<code>float</code>
<code>float + float</code>	<code>float</code>

! **int only stores whole numbers** — nothing else. If you want a correct/precise answer, use at least one float number in the equation (e.g. `3.0`, `5.0`, `6.0`).

• Typecasting

If we want to force-convert a float into an integer:

```
int n = 20;
float m = 33.5;

n = (int) m; // n now becomes 33 (decimal part dropped)
```

Relational Operators

Operator	Meaning
==	Equal to
<=	Less than or equal to
>=	Greater than or equal to
>	Greater than
<	Less than
!=	Not equal to

Logical Operators

In C, `0 = false` and `1 = true`.

Operator	Name	Rule
&&	AND	Both must be true: 0,1 → 0 1,1 → 1
	OR	At least one must be true: 1,0 → 1
!	NOT	Flips the value: a=1 → !a=0

REAL-LIFE EXAMPLE — &&

To enter an exam hall you need: (1) Student ID **AND** (2) Admit Card. Both are needed — if either one is missing, you won't get in. That's exactly how `&&` works.

REAL-LIFE EXAMPLE — ||

To log in somewhere you can use: (1) Email ID **OR** (2) Phone number. You can use just one of them — it works even with only one input being correct. That's exactly how `||` works.

  **just reverses the whole thing** — true becomes false, and false becomes true.

Ternary Operator — Shorthand if-else

```
condition ? expression-if-true : expression-if-false;
```

switch — kind of like if, but different

In `switch`, we can make several cases. When the entered number matches a case number, it prints that case **and every case below it too** (because it keeps falling through). To stop after just one case, we use `break` inside it.

There are 3 kinds of loops: **while**, **do-while**, and **for**.

S.S.C → **Start, Stop, Change** (applies to every loop, in some form)

while

Checks the condition *first*. If `i<4`, it prints the code as long as `i` stays less than 4.

```
while (i<4) {
    printf("%d",i);
    i++;
}
```

do-while

Same as while, but the difference is it **executes first, then checks** the condition.

```
do {
    printf("%d",i);
    i++;
} while(i<4);
```

for

Includes all 3 conditions to meet at once.

```
for(i=0;i<4;i++){
    printf("%d",i);
}
```

I.C.I → **Initialize, Condition test, Increment** (comma is mostly used here!)

• Increment / Decrement — `i++` vs `++i`

★ `i++` and `++i` are NOT the same!

Operator	Name	Behaviour
<code>i++</code>	Post-increment	Print first, then add 1 to the number
<code>++i</code>	Pre-increment	Add 1 to the number first, then print

• break & continue

break

When used, it will exit the loop right now — even skipping the remaining iterations.

continue

When used, it will skip just the current step in the loop — the iteration is just skipped, not the whole loop.

• Three Classic Loop Problems

1. Sum & Product of numbers

Sum usually **starts with 0**: `sum = 0; sum = sum + i;`

Product usually **starts with 1**: `product = 1; product = product * i;`

2. Factorial

Factorial means the multiplication of numbers until that number arrives.

`8! = 1 × 2 × 3 × 4 × 5 × 6 × 7 × 8`

3. Reverse a Number & Prime Check

`reverse = reverse * 10 + digit;` — repeat till the number changes down to a single digit (i.e. becomes 0).

Prime number — a number divisible only by 1 and itself. Examples of primes: 2, 3, 5, 7... Numbers with more than two factors are "not prime." **0 and 1 are not prime.**



Practice-set hint: "print 3 inside → do we want the loop repeated?" then the iteration is just a step among many. "print 3 outside → do we want only one output?" then it's a single step outside the loop.

Functions & Recursion

A **function** is a block of code which performs a specific/particular task.

Function Prototype

Tells the compiler *what* is about to happen — a little twist: it decides the core of the task, but doesn't give it yet.

Function Declaration

Where we actually decide the core of the task — and give a **return value**. If we don't need one, we use `void`.

Term	Meaning
Functions	Library functions (already given) or User-defined functions (we write)
Parameters	The placeholder — values are just variables that hold a place
Arguments	The actual value passed in

✔ **Change function analogy:** Arguments only pretend to change the value inside the function — but the function only sends a *copy*. The original value never changes (unless we use pointers — see Chapter 6).

• Math Library Functions

For maths we use `<math.h>` — it includes `pow(base, exponent)` and `sqrt()`. `pow` stores floats/doubles always, so we have to use `%f`.

```
pow(5, 3);    // 5 cubed
pow(2, 4);    // 2 to the power 4
```

• Recursion

Recursion is a function that calls itself. It is not often used, but it's important.

★ **The condition in a recursive function that stops further recursion is called the base case.** (V.V.I — very very important)

- **You have to make your own equation** in recursion.
- **In recursion, we need to call the function** — this is mainly for declaration.

• Recursion Cheat Sheet

Problem	Recursive Operation	Base Return
Sum	+	0
Product	*	1
Factorial	*	1
Count	1 +	0
Fibonacci	+	needs 2 known base cases
Power	*	1

✔ The **return value** will be a number that won't ruin your final result for the base case.

Pointer — a variable that stores the address of another variable. Think of it as **[Ahh] → Address holder**.

THE HOUSE ANALOGY

Imagine it like this:

You have a **house** → **x** (this means house x).

Inside the house → **10** (contains 10 in it).

Address of the house → **500** (this 500 is the pointer).

```
int *p = &x;      // Here x = 10 (the value)

&x = Address of x = 500
p   = Address of x = 500
*p  = Goes to the address and sees inside it = 10
```

& — "Give address"

Gives the address of a variable.

* — "Go to that address"

Goes to that address and sees inside it (the value).

★ **Magic part:** `*p = 50;` → goes to x and changes its value to 50. `&` and `*` can cut/undo each other equally.

• Pointer to Pointer

```
int i = 5;
int *p = &i;
int **k = &p;
```

Expression	Meaning
<code>p</code>	Stores the address of i
<code>k</code>	Stores the address of p
<code>*k</code>	Stores the value of p
<code>**k</code>	Stores the value of i

• Call by Value vs Call by Reference

Type	What happens
Call by value	The function receives a copy of the variable, so the original value does not change.
Call by reference	The function receives the address of the value, so the original value <i>can</i> change.

THE WALLET ANALOGY

Call by value: You give your friend a copy of your money. Even if he cuts or changes it, your original money won't be affected.

Call by reference: You give your friend your actual wallet. So if he takes or changes the money now, your original money will be affected.

What we need	Specifier to use
Pointer's address	<code>%p</code> or <code>%u</code>
Pointer's value	<code>%d</code>

✓ In pointers, we will mostly use `void` functions.

Array — a collection of same-type variables stored in continuous memory. It takes memory in location by bytes.

```
int arr[5];    // stores 5 integers
```

• Indexing Rule

Array index always starts from **0**. Last valid index = `size - 1`.

Index	0	1	2	3	4
Array	10	20	30	40	50

```
int arr[5] = {10, 20, 30, 40, 50};
// or, without size:
int arr[] = {10, 20, 30, 40, 50};
```

★ **Key relationship:** `arr[i]` is exactly the same as `*(arr + i)`. Pointer movement uses `ptr++`.

• 2-D Arrays — like a table or a matrix

```
int arr[3][3] = {
    {1, 2, 3},
    {4, 5, 6},
    {7, 8, 9}
};
// row → outer loop, column → inner loop
```

`int arr[3][3]` means 3 rows, 3 columns.

• Arrays & Functions

```
void printArray(int arr[], int n);
```

! When you pass an array to a function in C:

- It is **not** actually passed as a full array.
- It's automatically converted into a **pointer to the first element**.
- You don't write the size inside `[]` — that's because the array is already created in `main()`. The function just receives it.
- The size comes in separately, as its own parameter `n`.

✓ We mostly use a **for loop** with arrays — for taking/storing input, for printing output, and in many other ways. In arrays, we first **store**, then **print**.

String — a character array that ends with `'\0'` (the null character).

```
char str[] = "Avi";  
// in memory: A  V  i  \0
```

• String Initialization — Two Methods

Method 1
`char st[] = {'A','V','i','\0'};`

Method 2
`char st[] = "AVI";`

• Accessing & Printing

Accessing a character: `str[0] = A`, `str[1] = V`.

Goal	Code
Print a single character	<code>printf("%c", str[i]);</code>
Print the whole string	<code>printf("%s", str);</code>

• Taking Input

```
scanf("%s", str);
```

 `scanf("%s", ...)` only takes **one word** — it stops at the first space. So "Harry" works, but "Harry Potter" gets cut off. **scanf doesn't work with spaces.**

• Multi-word Input

Function	Purpose
<code>gets(str)</code>	Takes multiple words with spaces
<code>puts(str)</code>	Prints multiple words and moves the cursor to the next line, like <code>\n</code>

• Array vs Pointer Strings

Type	Example	Can it change?
Array string	<code>char str[] = "Avi";</code>	"Married" — cannot be reassigned later
Pointer string	<code>char *ptr = "Avi";</code>	Single — assignable, e.g. <code>ptr = "Harry";</code>

• String Functions — "The Fantastic Four" `#include <string.h>`

#	Function	Purpose
1	<code>strlen("Harry")</code>	Calculates length → returns 5
2	<code>strcpy(dest, src)</code>	Copies one string into another
3	<code>strcat(s1, s2)</code>	Joins two strings together
4	<code>strcmp(s1, s2)</code>	Compares two full strings

strlen
Counts the length **without** the null character.
Example: `"Harry" → 5`

strcmp
Can **only** be used for full strings — not for copying or comparing character by character.

- `strcmp("Avi", "Avi")` → 0 (equal)
- `strcmp("Avi", "Biswas")` → negative
- `strcmp("Biswas", "Avi")` → positive

Structure — a user-defined data type that allows us to store different types of related data under one name.

Example: Employee → code, salary, name. Student → roll, marks, name.

Syntax

```
struct employee {
    int code;           // this is a data type, not a variable
    float salary;
    char name[20];
}; // the semicolon is mandatory!

struct employee e1, e2; // these are variables
```

Accessing Members — the dot (.) operator

```
e1.code = 100;
e1.salary = 50000;
strcpy(e1.name, "Avi");
```

Structure Initialization

Instead of assigning line by line, we can write it directly — following the order the members were declared in (code, salary, name):

```
struct employee e1 = {100, 50000, "Avi"};
```

Array of Structures

```
struct employee emp[100];
// emp[0] → first employee
// emp[1] → second employee
// emp[2] → third employee
```

Pointer to Structures

```
struct employee *ptr;
ptr = &e1; // or: ptr = &e1;

// Accessing members using a pointer:
(*ptr).code // or, the cleaner way:
ptr->code
```

typedef with Structure

```
typedef struct employee {
    int code;
    float salary;
    char name[20];
} Employee;

// Now we can write directly:
Employee e1;
Employee e2;
```

File — a collection of data stored permanently on a storage device.

• File Pointer & Opening a File

```
FILE *ptr;
ptr = fopen("file.txt", "r/w");
      ↑ filename  ↑ mode
```

• Modes

Mode	Meaning
r	Read file
w	Write file (overwrites old content)
a	Append data at the end
rb	Read binary file
wb	Write binary file

• Reading & Writing

Reading from a file

```
fscanf(ptr, "%d", &num);
```

Writing to a file

```
fprintf(ptr, "%d", num);
```

• Character-by-Character I/O

Function	Purpose
fgetc(ptr)	Reads one character from a file
fputc(ptr)	Writes one character to a file

★ **EOF:** `fgetc()` returns `EOF` when the end of the file is reached.

✓ **Checking if a file exists:** `ptr == NULL` checks whether the file actually exists.

• Closing a File

```
fclose(ptr);
```

Dynamic Memory Allocation (DMA)

WHAT IS DMA?

DMA means adding memory **while the program is running**. It changes the fixed values of arrays — e.g. `int arr[5]` can effectively grow to `int arr[6]`, `[7]` or `[8]` using DMA functions.

• The Four DMA Functions

#	Function	Purpose
1	<code>malloc()</code>	Memory Allocation
2	<code>calloc()</code>	Contiguous Allocation
3	<code>realloc()</code>	Changes size of previously allocated memory
4	<code>free()</code>	Releases the allocated memory

• 1. malloc()

```
int *ptr;
ptr = (int*) malloc(5 * sizeof(int));
// this allocates memory for 5 integers
```

 **NOTE:** The memory allocated by malloc contains **garbage values** to start with.

• 2. calloc()

```
int *ptr;
ptr = (int*) calloc(5, sizeof(int));
```

 **NOTE:** The memory allocated by calloc contains values of **0** — already initialized, unlike malloc.

• 3. realloc()

To change the size of previously allocated memory:

```
ptr = realloc(ptr, new_size);
// example:
ptr = (int*) realloc(ptr, 10 * sizeof(int));
```

• 4. free()

To release the DMA memory after using it:

```
free(ptr);
```

• Summary

Function	Effect
<code>malloc()</code>	Allocates memory with garbage values
<code>calloc()</code>	Allocates memory and initializes it with 0
<code>realloc()</code>	Changes the size of already allocated memory
<code>free()</code>	Releases allocated memory